Kube-Proxy: What is it and How it Works



Amr Essam · Follow
11 min read · Jul 1

Listen Share ••• More



Photo by Jordan Harrison on Unsplash

Introduction

Networking is a crucial part of Kubernetes. Understanding how different network components work helps you design and configure them correctly for your application.

Behind the Kubernetes network, there is a component that works under the hood. It translates your Services into some usable networking rules. This component is called **Kube-Proxy**.

In this article we are going to show how Kube-Proxy works. We'll explain the flow that happens when creating Services. And we're going to show some example rules that Kube-Proxy creates.

If you're not familiar with basic Kubernetes networking, I recommend going through this blog first.

What is Kube-Proxy

We know that Pods are ephemeral in Kubernetes. They can be terminated or restarted at any time. Due to this behavior, we cannot rely on their IP addresses as they always change.

Here's where the **Service** object comes into play. Services provide a stable IP address to connect to Pods. Each Service is associated with a group of Pods. When traffic reaches the Service, it is redirected to the backend Pods accordingly.

But how actually is this "Service to Pod" mapping implemented on a networking level ?

That's the role of the Kube-proxy.

Kube-Proxy is a Kubernetes agent installed on **every node** in the cluster. It monitors the changes that happen to Service objects and their endpoints. Then it translates these changes into actual network rules inside the node.

Kube-Proxy usually runs in your cluster in the form of a **DaemonSet**. But it can also be installed directly as a Linux process on the node. This depends on your cluster installation type.

If you use kubeadm, it will install Kube-Proxy as a DaemonSet. If you manually install the cluster components using official Linux tarball binaries, it will run directly as a process on the node.

How Kube-Proxy Works

After Kube-proxy is installed, it authenticates with the API server. When new Services or endpoints are added or removed, the API server communicates these changes to the Kube-Proxy.

Kube-Proxy then applies these changes as **NAT** rules inside the node. These NAT rules are simply mappings from Service IP to Pod IP.

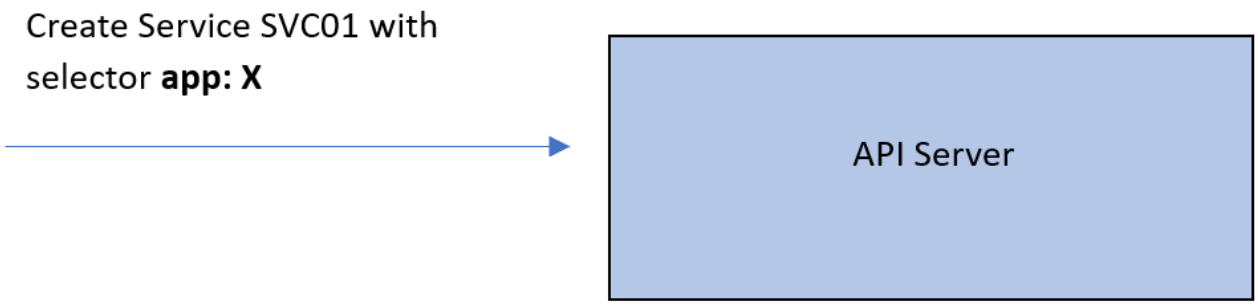
When traffic is sent to a Service, it is redirected to a backend Pod based on these rules.

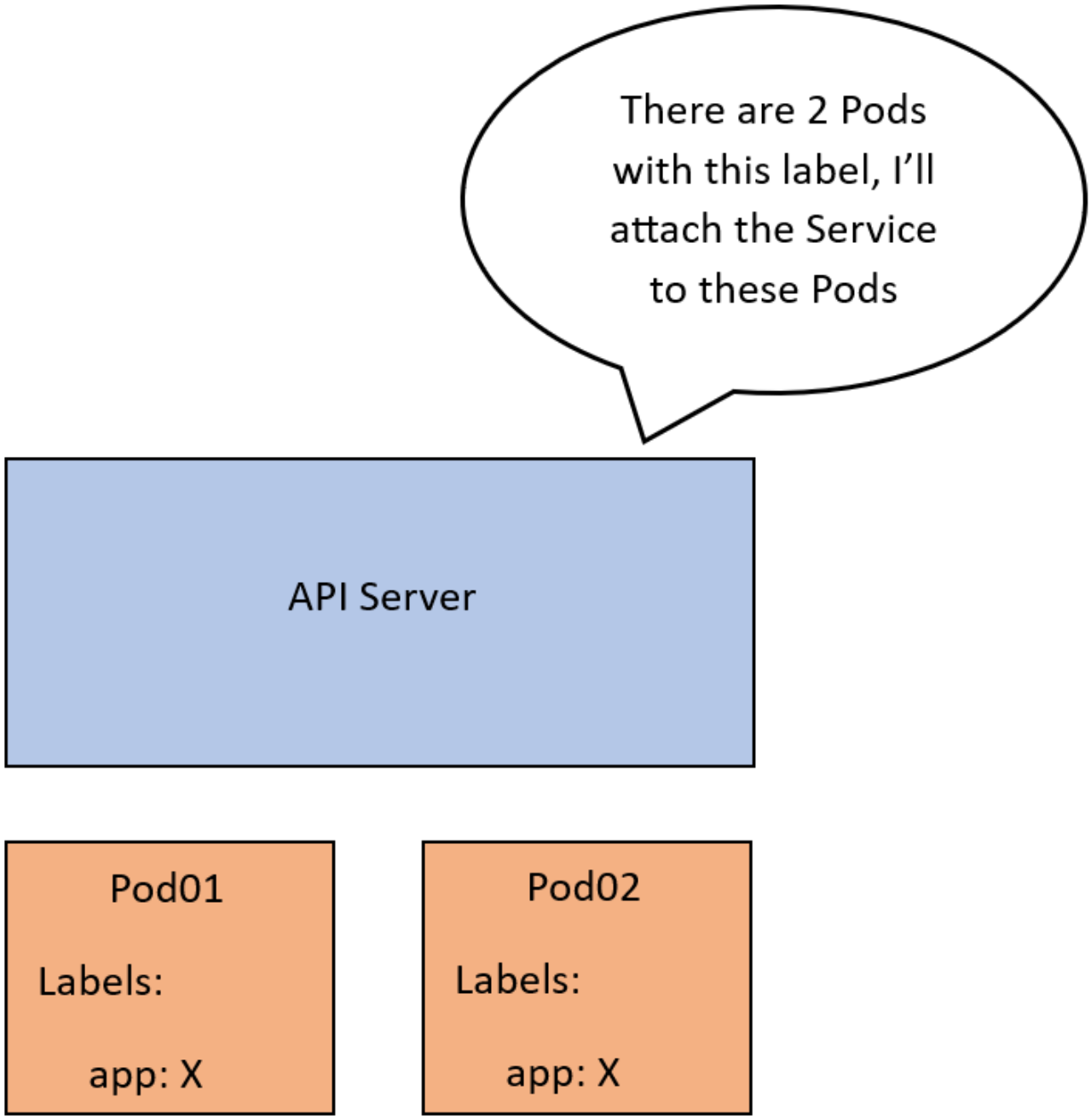
Now let's get into more details with an example.

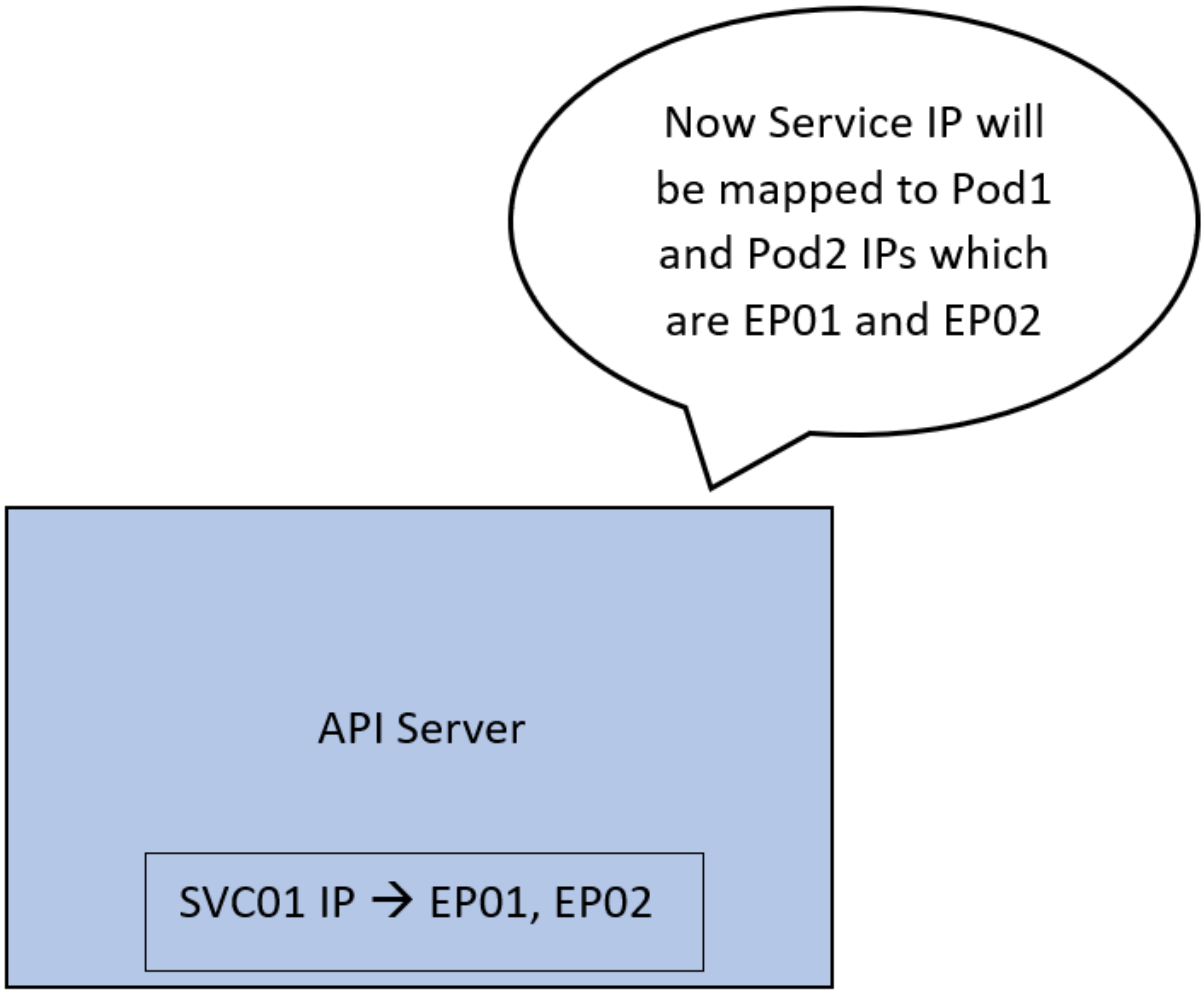
Assume we have a Service **SVC01** of type ClusterIP. When this Service is created, the API server will check which Pods to be associated with this Service. So, it will look for Pods with **labels** that match the Service's **label selector**.

Let's call these **Pod01** and **Pod02.** Now the API server will create an abstraction called **endpoint**. Each endpoint represents the IP of one of the Pods. **SVC01** now is tied to 2 endpoints that correspond to our Pods. Let's call these **EP01** and **EP02.**

Now the API server maps the IP address of **SVC01** to 2 IP addresses **EP01** and **EP02.**

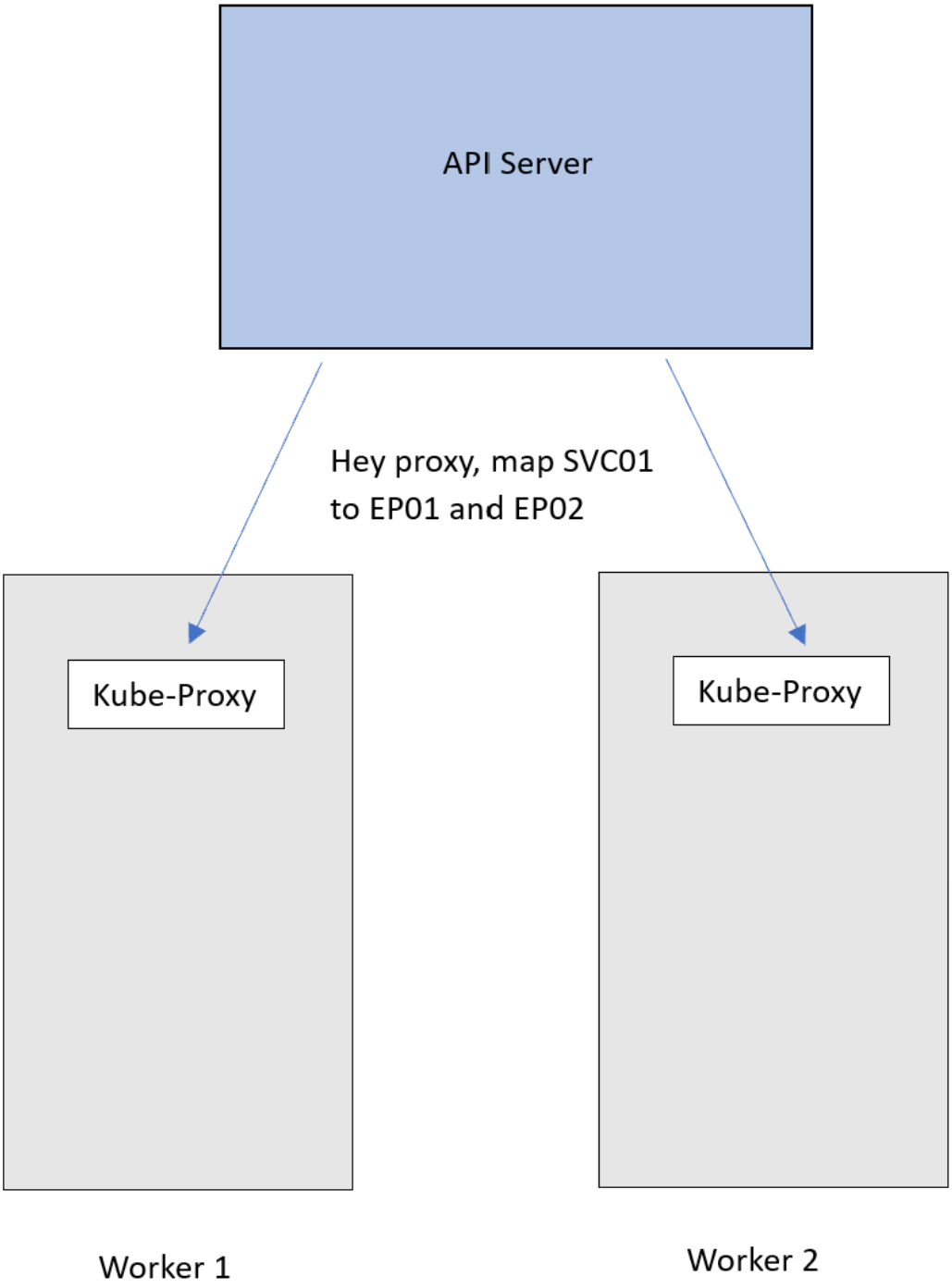


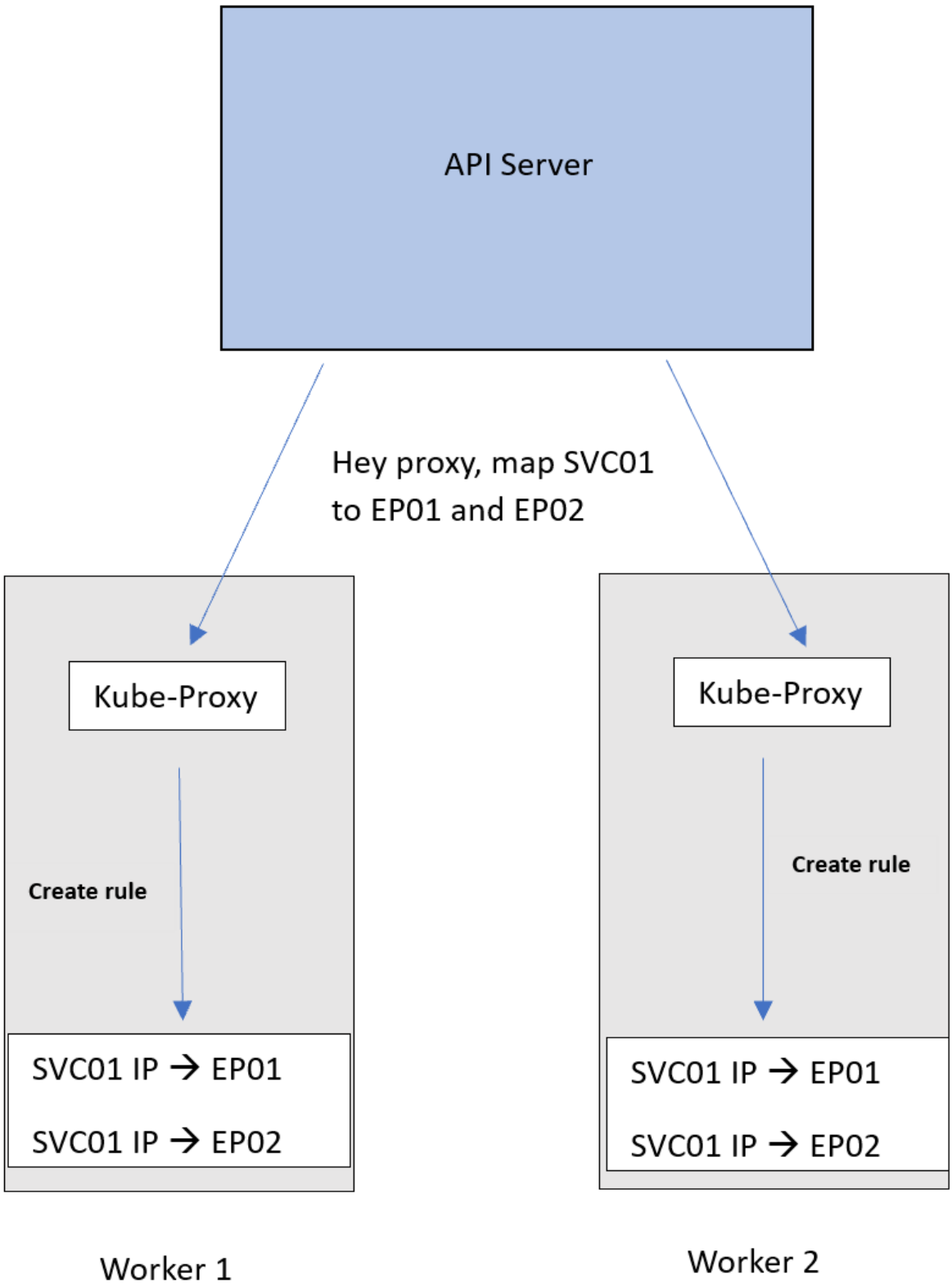

There are 2 Pods with this label, I'll attach the Service to these Pods
API Server
Pod01
Labels:
app: X
Pod02
Labels:
app: X



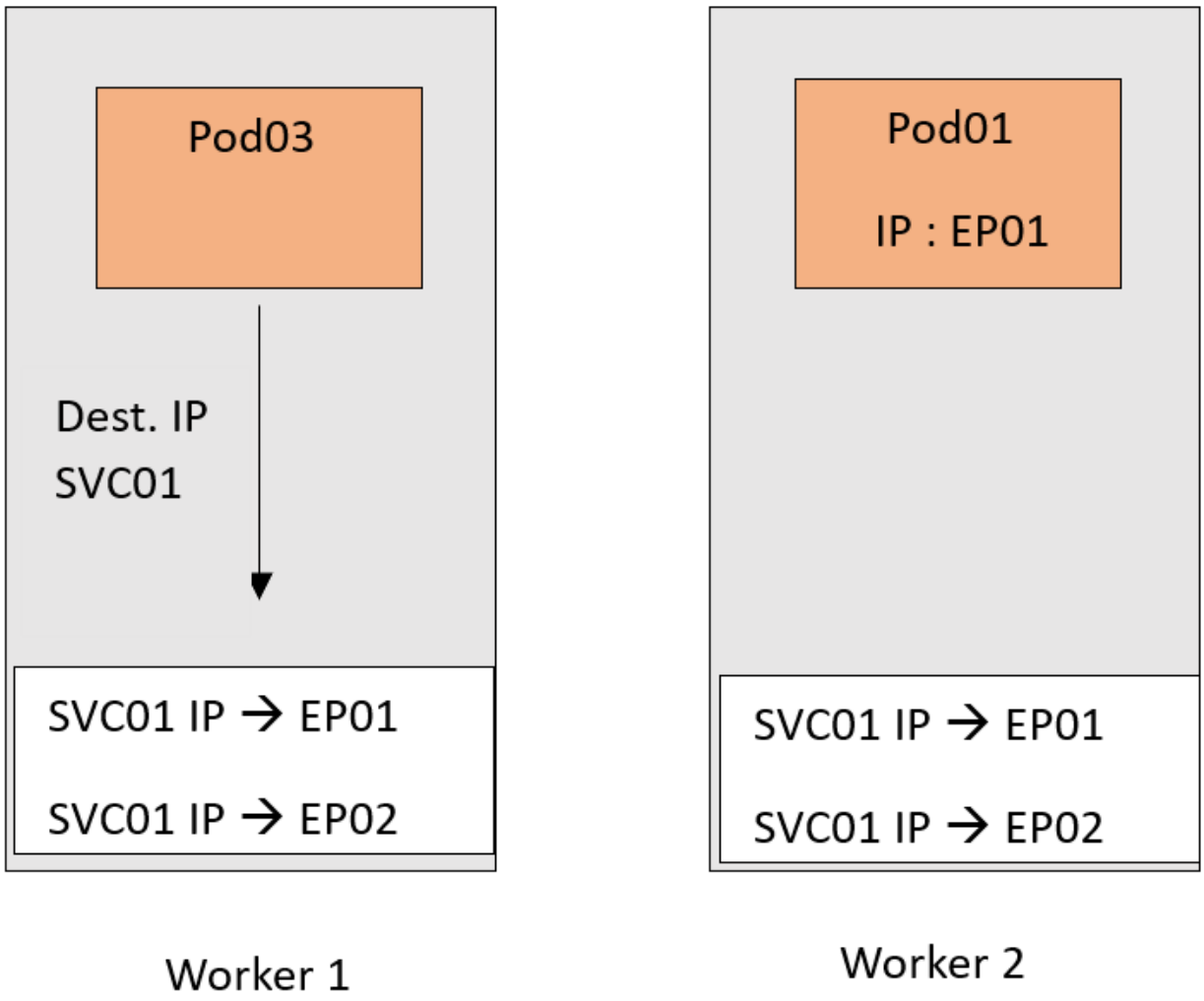
All of this configuration is currently only part of the control plane. We want this mapping to be actually implemented on the network. Once it is applied, traffic coming to the IP of **SVC01** will be forwarded to **EP01** or **EP02.**

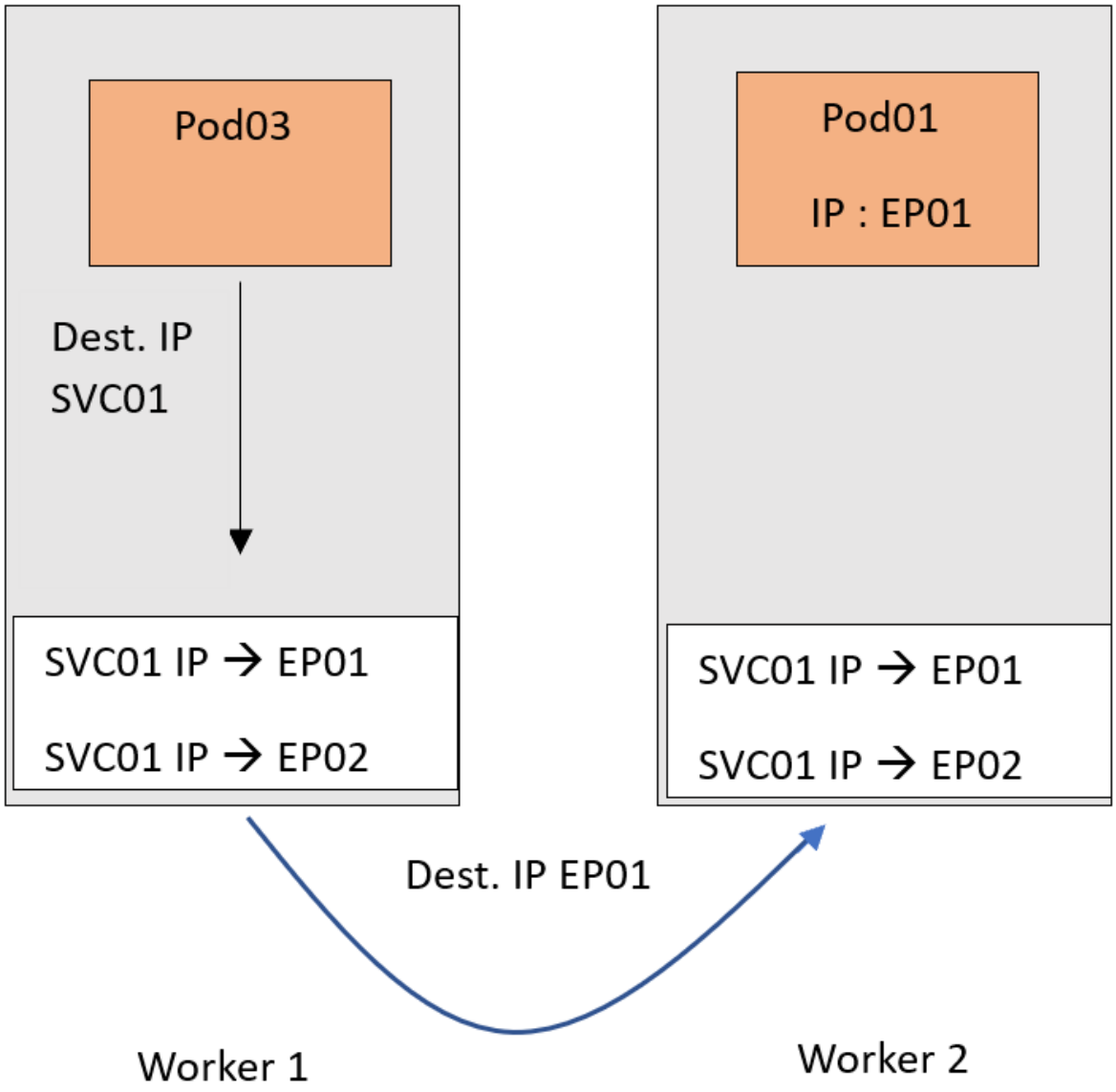
So here comes the Kube-Proxy. The API server will advertise these updates to the Kube-proxy on each node. Which will apply it as internal rules to the node.


API Server
Hey proxy, map SVC01
to EP01 and EP02
Kube-Proxy
Kube-Proxy
Worker 1
Worker 2



Now traffic destined to the SVC01 IP will follow this **DNAT** rule and get forwarded to the Pods. Remember that EP01 and EP02 are basically the IPs of the Pods.


Pod03
Dest. IP
SVC01
SVC01 IP → EP01
SVC01 IP → EP02
Worker 1
Pod01
IP : EP01
SVC01 IP → EP01
SVC01 IP → EP02
Worker 2



Now of course I tried to keep the scenario as simple as possible. This is just to focus on the important part of the Kube-Proxy. However, there are a couple of points that are worth mentioning :

1- The Service and endpoint are an **IP and Port** mappings, and not only an IP.

2- The DNAT translation in the example happened on the **source node.** This is because we used a **ClusterIP** type of service. That's the reason why ClusterIP is never routed outside the cluster. It's only accessible from inside the cluster as it's basically an internal NAT rule. In other words, no one outside the cluster knows about this IP.

3- If another type of service is used, other rules are differently installed inside the nodes. They might be placed separately in what is called a **chain**. Although this is out of the scope of the main topic, chains are sets of rules inside Linux machines. They have a specific type and are applied in a specific order in the traffic path.

4- The NAT rules pick one of the Pods **randomly.** However, this behavior might change depending on the Kube-Proxy "mode".

And that's our next topic to discover.

Kube-Proxy modes

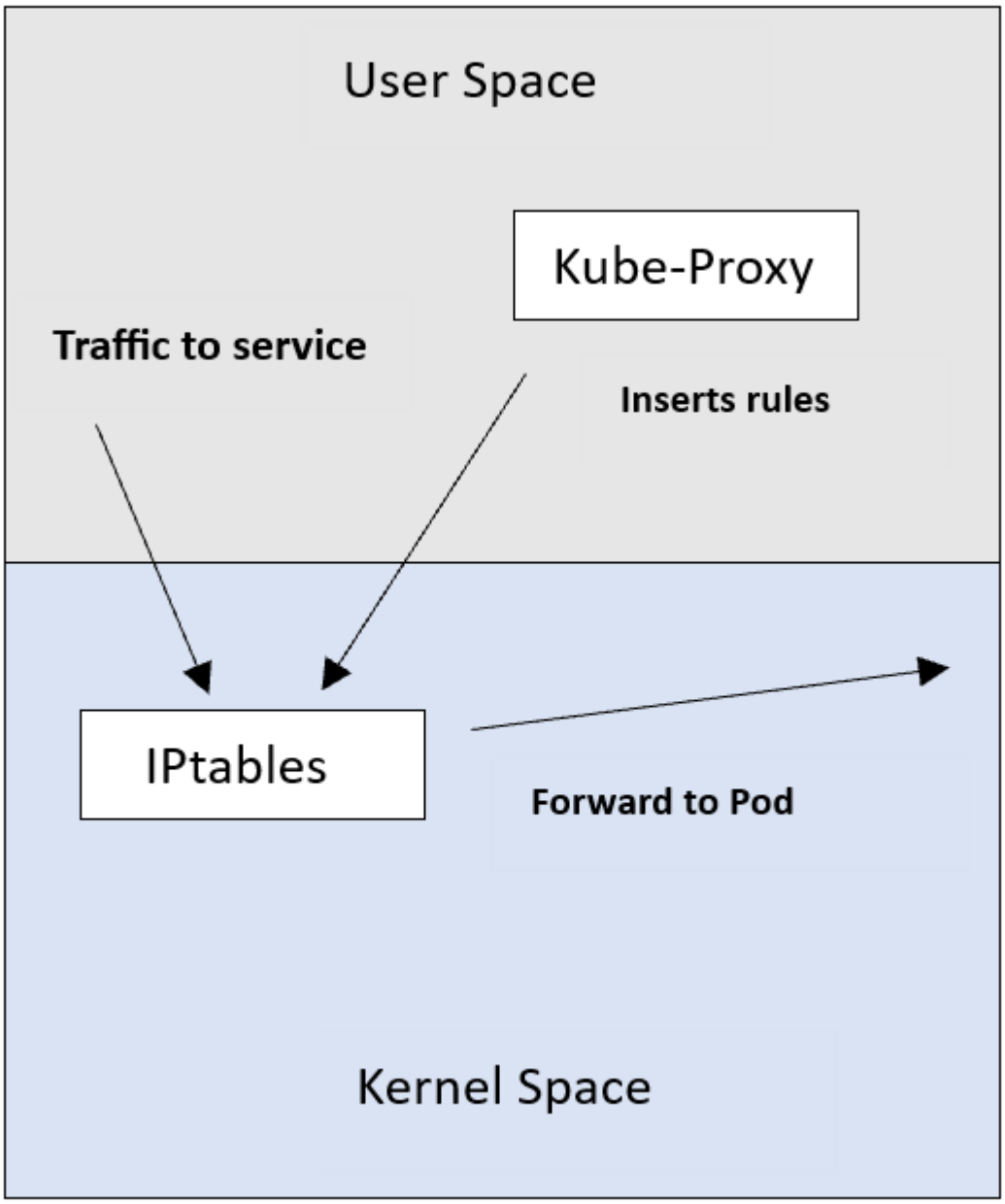
Kube-Proxy can operate in different modes. Each mode decides how Kube-Proxy implements the NAT rules we mentioned. With each mode having its pros and cons, we need to understand how they work.

IPtables mode

This is the default and most widely used mode today. In this mode Kube-Proxy relies on a Linux feature called **IPtables**. IPtables works as an internal packet processing and filtering component. It inspects incoming and outgoing traffic to the Linux machine. Then it applies specific rules against packets that match specific criteria.

When running in this mode, Kube-proxy inserts the Service-to-Pod NAT rules in the IPtables. By doing this, traffic is redirected to the respective backend Pods after the destination IP is **NATed** from the Service IP to the Pod IP.

Now the Kube-Proxy role can be described more as the "installer" of the rules.



The downside of this mode is that IPtables uses a sequential approach going through its tables. Because it was originally designed as a packet filtering component.

This sequential algorithm is not suitable when the number of rules increases. In our scenario, this will be the number of Services and endpoints. Taking this at a low level, the algorithm will follow O(n) performance. Which means that the number of lookups increases linearly by increasing the rules.

IPtables also doesn't support specific load balancing algorithms. It uses a random equal-cost way of distribution as we mentioned in our first example.

IPVS mode

IPVS is a Linux feature designed specifically for load balancing. This makes it a perfect choice for Kube-Proxy to use. In this mode, Kube-Proxy inserts rules into IPVS instead of IPtables.

IPVS has an optimized lookup algorithm with complexity of O(1). Which means that regardless of how many rules are inserted, it provides almost a consistent performance.

In our case, this means a more efficient connection processing for Services and endpoints.

IPVS also supports different load balancing algorithms like round robin, least connections, and other hashing approaches.

Despite its advantages, IPVS might not be present in all Linux systems today. In contrast to IPtables which is a core feature of almost every Linux operating system.

Also if you don't have that many Services, IPtables should work perfectly.

KernelSpace mode

This mode is specific to Windows nodes. In this mode Kube-proxy uses Windows *Virtual Filtering Platform (VFP)* to insert the packet filtering rules. The ***VFP*** on Windows works the same as IPtables on Linux, which means that these rules will also be responsible for rewriting the packet encapsulation and replacing the destination IP address with the IP of the backend Pod.

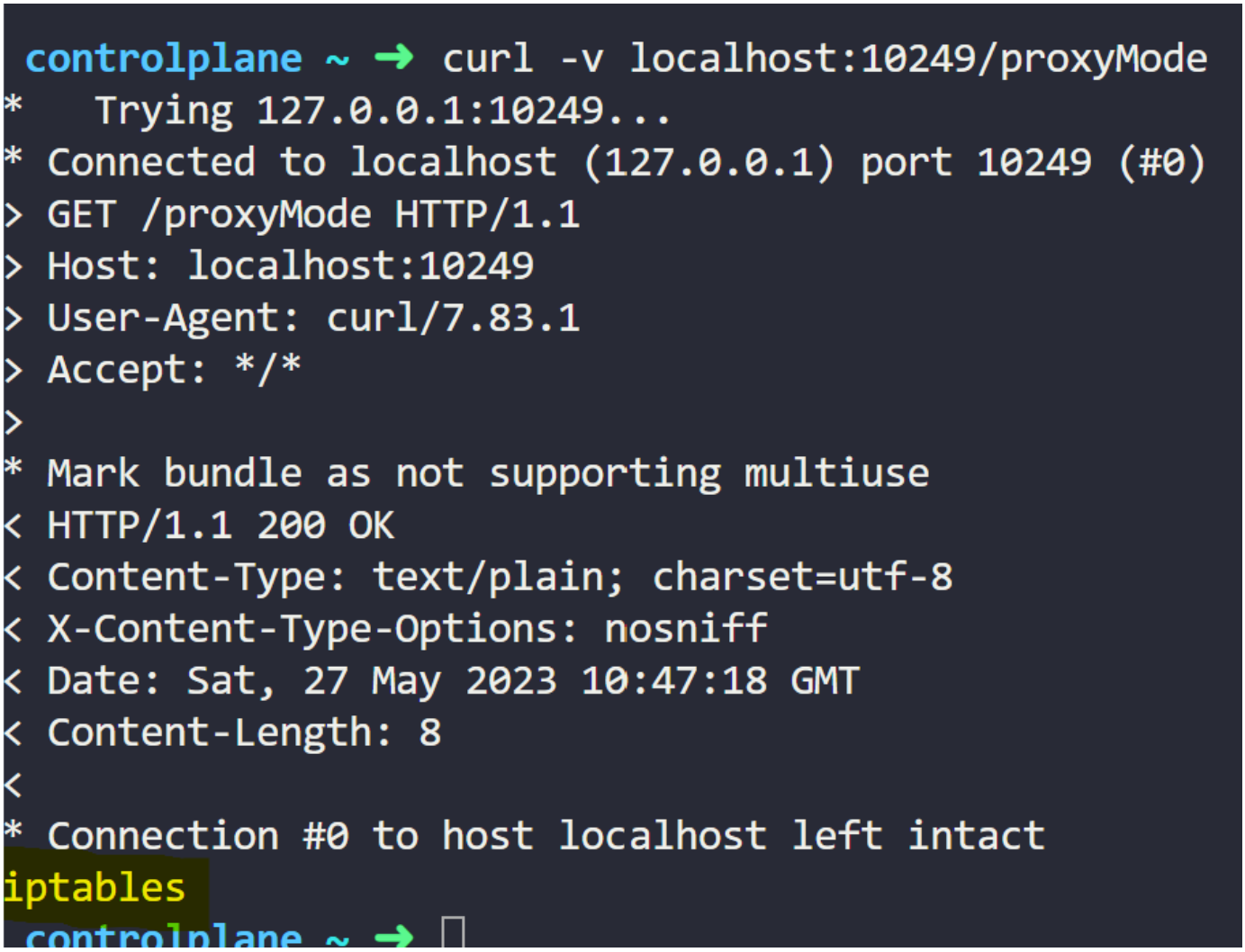
If you're familiar with virtual machines, specifically on Windows platform, so you can think of ***VFP*** as an extension of the Hyper-V switch which was originally used for virtual machine networking.

How to check Kube-Proxy mode

By default, Kube-proxy runs on port 10249 and exposes a set of endpoints that you can use to query Kube-proxy for information.

You can use the /proxyMode endpoint to check the kube-proxy mode.

First connect through SSH to one of the nodes in the cluster. Then use the command **curl -v localhost:10249/proxyMode.**



Here you can see that Kube-Proxy is using iptables mode.

Checking IPtables rules for ClusterIP service

Now let's inspect the IPtables rules with a closer look. We'll create a ClusterIP service and check the rules created.

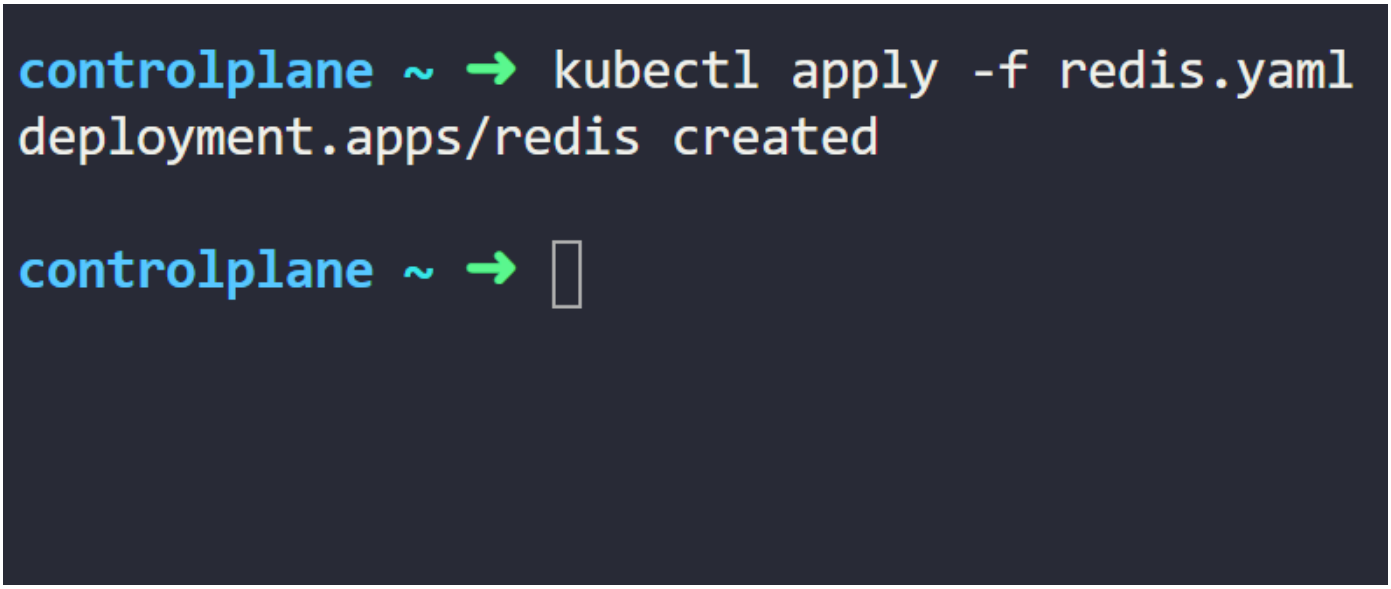
Prerequisites:

- A working Kubernetes cluster (single or multi-node)
- Kubectl installed to connect to the cluster and create the required resources
- SSH enabled to one of the nodes where we'll check the rules

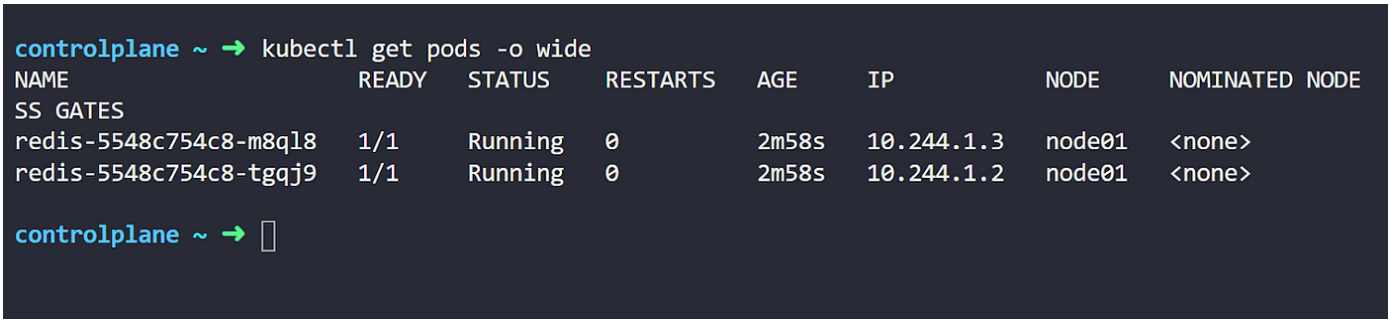
Steps:

- First we'll create a redis deployment with 2 replicas.

```
controlplane ~ ➜ cat redis.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: redis
  labels:
    app: redis
spec:
  replicas: 2
  selector:
    matchLabels:
      app: redis
  template:
    metadata:
      labels:
        app: redis
    spec:
      containers:
      - name: redis
        image: redis
        ports:
        - containerPort: 6379
          name: redis
```



Now let's check the Pods created.



You can see we have 2 running Pods with their IP addresses.

- Let's create a Service that is attached to these Pods. To do this we create the Service with a selector matching the Pods labels.

```
controlplane ~ ➜ cat redis-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: redis
spec:
  selector:
    app: redis
  ports:
    - protocol: TCP
      port: 6379
      targetPort: 6379
```

```
controlplane ~ ➜ kubectl apply -f redis-service.yaml
service/redis created

controlplane ~ ➜
```

Listing available Services, we can see our redis Service with its IP address.

```
controlplane ~ ➜ kubectl get svc
NAME         TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)    AGE
kubernetes   ClusterIP   10.96.0.1       <none>        443/TCP    9m40s
redis        ClusterIP   10.99.231.137   <none>        6379/TCP   34s

controlplane ~ ➜
```

Note that we didn't specify the Service type in the YAML manifest. This is because the default type is ClusterIP.

- Now if we list the endpoints, we'll find that our Service has 2 endpoints corresponding to our Pods.

```
controlplane ~ ➜ kubectl get ep
NAME         ENDPOINTS                                AGE
kubernetes   192.24.115.3:6443                        28m
redis        10.244.1.2:6379,10.244.1.3:6379          6m50s

controlplane ~ ➜
```

You'll notice that the 2 endpoints represent the IP addresses of the Pods.

So far all of this configuration is pretty much straight forward. Now let's explore the magic under the hood.

- We'll list the IPtables rules on one of the nodes. Note that you need to SSH into the node first to run the following commands.

```
controlplane ~ ➜ iptables -t nat -L PREROUTING
Chain PREROUTING (policy ACCEPT)
target     prot opt source               destination
KUBE-SERVICES  all  --  anywhere             anywhere             /* kubernetes service portals */
DOCKER_OUTPUT  all  --  anywhere             172.25.0.1

controlplane ~ ➜
```

Of course we'll not dive into all the details of IPtables here. We'll try to highlight only the important information for our scenario.

Let's explain our command options first. The "**-t nat**" refers to which type of table we want to list. IPtables include multiple table types, for Kube-Proxy it uses the NAT table. This is because Kube-Proxy uses IPtables mainly for translating Service IPs.

Remember when we mentioned the concept of a chain ? The **"-L PREROUTING"** here refers to the name of the chain in the table. This is a chain that exists by default in the IPtables. Kube-Proxy hooks its rules into that chain.

So briefly, here we want to list the nat rules inside the PREROUTING chain.

Now let's move to the command output. The most important part of the output here is the **KUBE-SERVICES** line. This is a custom chain created by Kube-Proxy for the Services. You'll notice that the rule forwards traffic with any source and any destination to this chain.

In other words, packets going through the PREROUTING chain will be directed to the KUBE-SERVICES chain.

So let's check what's inside this KUBE-SERVICES.

```
controlplane ~ ➜ iptables -t nat -L KUBE-SERVICES
Chain KUBE-SERVICES (2 references)
target     prot opt source               destination
KUBE-SVC-JD5MR3NA4I4DYORP  tcp  --  anywhere             10.96.0.10           /* kube-system/kube-dns:metr
s cluster IP */ tcp dpt:9153
KUBE-SVC-OKJCEJEOAS2LLIDR  tcp  --  anywhere             10.99.231.137        /* default/redis cluster IP
 tcp dpt:6379
KUBE-SVC-NPX46M4PTMTKRN6Y  tcp  --  anywhere             10.96.0.1            /* default/kubernetes:https
uster IP */ tcp dpt:https
KUBE-SVC-TCOU7JCQXEZGVUNU  udp  --  anywhere             10.96.0.10           /* kube-system/kube-dns:dns
uster IP */ udp dpt:domain
KUBE-SVC-ERIFXISQEP7F7OF4  tcp  --  anywhere             10.96.0.10           /* kube-system/kube-dns:dns-
p cluster IP */ tcp dpt:domain
KUBE-NODEPORTS  all  --  anywhere             anywhere             /* kubernetes service nodeports; NOTE:
is must be the last rule in this chain */ ADDRTYPE match dst-type LOCAL
```

Now what's this cryptography here? Well, simply these are a couple more chains.

The thing that matters to us here is the IP addresses. You'll notice a specific chain created with a rule for destination IP of the Service (10.99.231.137). This means that traffic destined to the Service will enter that chain.

You can also see some information to the right of the IP address. These represent the Service name, Service type, and port.

All of this ensures that we're looking at the correct section of IPtables. Now let's get into this chain.

```
controlplane ~ ➜ iptables -t nat -L KUBE-SVC-OKJCEJEOAS2LLIDR
Chain KUBE-SVC-OKJCEJEOAS2LLIDR (1 references)
target     prot opt source               destination
KUBE-MARK-MASQ  tcp  -- !10.244.0.0/16        10.99.231.137        /* default/redis cluster IP */ tcp dpt:63
79
KUBE-SEP-7PFXXPBHE4AJG7XF  all  --  anywhere             anywhere             /* default/redis -> 10.244.1.2
:6379 */ statistic mode random probability 0.50000000000
KUBE-SEP-I4DTWEFCQUPMAXQX  all  --  anywhere             anywhere             /* default/redis -> 10.244.1.3
:6379 */
```

And finally here we've arrived at our NAT rules. You'll find 2 additional chains created. Each starts with **KUBE-SEP** and then a random id. These correspond to the Service endpoints (SEP). You can see the IP address of each Pod listed for each chain.

Notice this **statistic mode random probability** line in the middle? This is the random load balancing rule that the IPtables makes between the Pods.

Going into any of these KUBE-SEP chains we can see it's basically a **DNAT** rule.

```
controlplane ~ ➜ iptables -t nat -L KUBE-SEP-7PFXXPBHE4AJG7XF
Chain KUBE-SEP-7PFXXPBHE4AJG7XF (1 references)
target     prot opt source               destination
KUBE-MARK-MASQ  all  --  10.244.1.2           anywhere             /* default/redis */
DNAT       tcp  --  anywhere             anywhere             /* default/redis */ tcp DNAT [unsupported rev
sion]
```

So that's it for the IPtables rules for the Service. As you now know how to dig deeper into this, you can start exploring more of these rules in your environment.

FAQ

Is a Kubernetes Service a Proxy ?

Yes, a Kubernetes Service works a lot like a proxy.

It provides a stable IP that clients can connect to. Traffic received on this IP is redirected to a backend Pod IP. This overcomes the problem of Pods' IPs being changed each time a Pod is recreated.

Can Kube-Proxy perform load balancing ?

It depends on which part of Kube-Proxy is meant.

If we're talking about the Kube-Proxy agent itself, the answer is no. The Kube-Proxy agent doesn't receive the actual traffic or do any load balancing. This agent is only part of the control plane that creates the Service rules.

If we're talking about the rules that Kube-Proxy creates, the answer is yes. Kube-Proxy

Open in app ↗

Search Medium

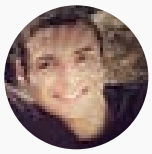
Conclusion

Kube-Proxy is a Kubernetes agent that translates Service definitions into networking rules. It runs on every node in the cluster and communicates with the API server to receive updates. These updates are then populated by the Kube-Proxy inside the node.

By creating these rules, Kube-Proxy allows traffic sent to a Service to be forwarded to the correct Pods. This enables the decoupling of Pod IP from clients connecting to it.

If you're interested to learn more about Kubernetes and its components you can check this quick Kubernetes tutorial.

Kubernetes Networking Kube Proxy Service Iptables



Follow

Written by Amr Essam

19 Followers

DevOps Engineer with 6+ Yrs of experience in Cloud, Virtualization, and DevOps technologies.